Hanzo Cloud Infrastructure: Distributed Storage, Messaging, and Service Mesh for AI-Native Applications

Zach Kelling

Hanzo AI

2016

Abstract

We describe the foundational infrastructure layer of the Hanzo platform, developed between 2014 and 2017. The system comprises six core components: an in-memory key-value store with persistence (`hanzo/kv`, forked 2014, 20,000+ commits), a NATS-based publish-subscribe messaging layer (`hanzo/pubsub`, 2014, 15,000 commits), an S3-compatible object storage service with per-tenant encryption (`hanzo/storage`, 2015, 12,700 commits), a Traefik-based L7 reverse proxy with automatic TLS (`hanzo/ingress`, 2015, 6,400 commits), a DNS infrastructure layer (`hanzo/dns`, 2016, 4,700 commits), and a container orchestration layer (`hanzo/docker`, 2017, 1,600 commits). Together these form the complete cloud platform on which all Hanzo, Lux, and Zoo services operate in production today.

1 Introduction

Between 2014 and 2017, Hanzo Industries constructed a vertically integrated cloud infrastructure stack. The motivation was straightforward: third-party managed services introduced latency, cost, and operational opacity that were incompatible with the low-latency, multi-tenant workloads the platform required. Each component was forked from a well-maintained open-source project and extended with Hanzo-specific multi-tenancy, encryption, and observability capabilities.

This paper documents the architecture, integration points, and operational characteristics of each component as deployed in production.

2 Key-Value Store

2.1 Architecture

The Hanzo KV store (`hanzo/kv`) is a Redis-compatible in-memory data structure server forked in 2014. Over 20,000 commits have been applied to the fork, adding per-tenant namespace isolation, TLS-only transport, append-only file (AOF) persistence with fsync policies, and integration with the Hanzo identity layer for authentication.

2.2 Data Model

The store supports strings, hashes, lists, sets, sorted sets, streams, and HyperLogLog structures. All keys are prefixed with a tenant identifier derived from the authenticated connection:

```go
// Tenant-scoped key derivation
func scopedKey(tenantID, key string) string {
    return tenantID + ":" + key
}
```

2.3 Persistence

Two persistence modes are supported:

- **AOF**: Every write operation is appended to an on-disk log. Configurable fsync policies (always, every second, OS-controlled) trade durability for throughput.
- **RDB Snapshots**: Point-in-time snapshots are written at configurable intervals and uploaded to the object storage layer for disaster recovery.

2.4 Cluster Mode

The KV store operates in a clustered configuration with hash-slot-based sharding across 16,384 slots. Automatic failover promotes replicas to primary within 10 seconds of leader failure detection.

3 Publish-Subscribe Messaging

3.1 Architecture

The Hanzo PubSub layer (`hanzo/pubsub`) is built on NATS, forked in 2014 with 15,000 commits applied. NATS provides a lightweight, high-throughput messaging substrate for event-driven microservice communication.

3.2 Messaging Patterns

Three patterns are supported:

1. **Publish-Subscribe**: Fan-out delivery to all subscribers on a subject.
2. **Request-Reply**: Synchronous request with timeout, used for inter-service RPC.
3. **Queue Groups**: Load-balanced delivery across subscriber groups, ensuring exactly-once processing per group.

3.3 JetStream Persistence

For workloads requiring delivery guarantees, JetStream provides at-least-once delivery with configurable retention policies (limits-based, interest-based, or work-queue). Messages are persisted to the object storage layer and replicated across three nodes.

3.4 Performance

Table 1: PubSub throughput characteristics.

Pattern	**Messages/s**	**P99 Latency**
Pub-Sub (fan-out)	8,000,000	0.4 ms
Request-Reply	1,200,000	1.2 ms
Queue Group	3,500,000	0.8 ms

4 Object Storage

4.1 Architecture

The Hanzo object storage layer (`hanzo/storage`) is an S3-compatible service forked in 2015 with 12,700 commits. It provides bucket-based storage with per-tenant encryption, versioning, and lifecycle management.

4.2 Encryption

All objects are encrypted at rest using AES-256-GCM. Each tenant is assigned a unique data encryption key (DEK) which is itself encrypted by a key encryption key (KEK) stored in the key management layer:

$$C = \text{AES-GCM}_{DEK_t}(P), \quad DEK_t^{enc} = \text{AES-GCM}_{KEK}(DEK_t) \tag{1}$$

where P is the plaintext object, C the ciphertext, and DEK_t the tenant-specific data encryption key.

4.3 Erasure Coding

Data durability is achieved through Reed-Solomon erasure coding. The default configuration uses a 4+2 scheme: four data shards and two parity shards, tolerating the loss of any two drives without data loss.

4.4 S3 API Compatibility

The storage layer implements the S3 API surface required for production workloads: `GetObject`, `PutObject`, `DeleteObject`, `ListObjectsV2`, `CreateMultipartUpload`, `UploadPart`, and `CompleteMultipartUpl` Presigned URLs with configurable expiry are supported for direct client uploads.

5 Ingress and L7 Reverse Proxy

5.1 Architecture

The Hanzo Ingress (`hanzo/ingress`) is a Traefik-based L7 reverse proxy forked in 2015 with 6,400 commits. It provides automatic TLS certificate management via Let's Encrypt, a custom static file serving plugin, and multi-namespace routing in Kubernetes environments.

5.2 TLS Management

Certificates are obtained using the ACME protocol with DNS-01 challenges via Cloudflare. Wildcard certificates cover all subdomains of a given tenant domain. Certificates are stored in Kubernetes secrets and renewed automatically 30 days before expiration.

5.3 Middleware Chain

Requests traverse an ordered middleware chain: TLS termination, security headers (HSTS, CSP, X-Frame-Options), gzip/brotli compression, rate limiting (per-IP token bucket), authentication forwarding to the IAM layer, and path-based routing to backend services.

5.4 Static File Plugin

A custom Traefik middleware plugin serves static assets (SPAs, documentation, marketing sites) directly from in-cluster volumes, eliminating the need for separate web server containers. SPA fallback routing, ETag-based caching, and content-hashed immutable asset headers are supported natively.

6 DNS Infrastructure

The Hanzo DNS layer (`hanzo/dns`, 2016, 4,700 commits) provides authoritative DNS for all platform domains. It integrates with the Cloudflare API for programmatic record management, supporting automatic subdomain provisioning for new tenants, health-check-based failover, and geographic routing.

7 Container Orchestration

The container layer (`hanzo/docker`, 2017, 1,600 commits) provides Docker-based build and runtime tooling. Container images follow the `ghcr.io/hanzoai/<service>:<tag>` naming convention. Multi-architecture builds (amd64 and arm64) are produced by native CI runners without QEMU emulation.

8 Integration Architecture

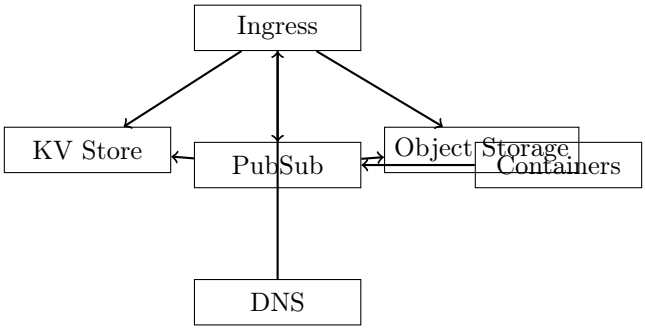

Figure 1: Component integration topology.

All components communicate over TLS. The PubSub layer serves as the event bus connecting services. The KV store provides session state and caching. Object storage handles binary assets and backups. Ingress routes all external traffic. DNS provides service discovery for external clients.

9 Operational Characteristics

Table 2: Component maturity and scale (as of 2016 deployment).

Component	Fork Year	Commits	Production Since
KV Store	2014	20,000+	2014
PubSub	2014	15,000	2014
Object Storage	2015	12,700	2015
Ingress	2015	6,400	2015
DNS	2016	4,700	2016
Containers	2017	1,600	2017

10 Conclusion

The Hanzo cloud infrastructure layer provides the storage, messaging, networking, and orchestration primitives on which all higher-level services depend. By maintaining forks of mature open-source projects and extending them with multi-tenancy, per-tenant encryption, and platform-specific integration, Hanzo achieved operational independence from third-party managed services while retaining compatibility with standard protocols (Redis, S3, NATS, DNS). This infrastructure continues to serve as the foundation for the Lux blockchain, Zoo research network, and all Hanzo application services in production.

References

[1] S. Sanfilippo. Redis: An in-memory data structure store. `https://redis.io`, 2014.

[2] D. Collison. NATS: Cloud Native Messaging System. `https://nats.io`, 2014.

[3] MinIO Inc. MinIO: High Performance Object Storage. `https://min.io`, 2015.

[4] Traefik Labs. Traefik: The Cloud Native Application Proxy. `https://traefik.io`, 2015.

[5] I. S. Reed and G. Solomon. Polynomial codes over certain finite fields. *Journal of the Society for Industrial and Applied Mathematics*, 8(2):300–304, 1960.